



I E S A L B A R R E G A S

TÉCNICO SUPERIOR DESARROLLO DE APLICACIONES
MULTIPLATAFORMA

Departamento de informática



Autor/es: Iván Rubio Murillo , Daniel Moñino García ,

Luis José Marcano Fuentes

Curso Académico: 2025-2026



“Proyecto de Desarrollo de Aplicaciones Multiplataforma”

ÍNDICE DEL PROYECTO: BOOK&CUT

1. INTRODUCCIÓN

- **1.1. Estructura del repositorio (Backend)**

2. ARQUITECTURA DE LA APLICACIÓN

- **2.1. Introducción**

3. BACKEND

- **3.1. Modelo**
- **3.2. Vista**
- **3.3. Controlador**

4. FRONTEND

5. DOCUMENTACIÓN TÉCNICA

- **5.1. Análisis**

6. DESARROLLO

- **6.1. Diseño de la BBDD - MySQL**

6.1.1. Normalización

6.1.2. Integridad referencial

6.1.3. Baja lógica

7. MODELO ENTIDAD / RELACIÓN DE LA BBDD

8. MODELO RELACIONAL DE LA BBDD

- **8.1. Decisiones de diseño**

9. DIAGRAMA DE CASOS DE USO

10. PRUEBAS

11. PROCESO DE DESPLIEGUE

12. PROPUESTAS DE MEJORAS

- **12.1.** Requisitos funcionales pendientes
- **12.2.** Requisitos técnicos

13. BIBLIOGRAFÍA

- **13.1.** Documentación Oficial (Spring.io, Flutter.dev, etc.).
 - **13.2.** Webs de referencia (StackOverflow, tutoriales utilizados).
-

1. Introducción

En el contexto actual, la transformación digital de los negocios de estética personal es una necesidad ineludible. **Book&cut** nace como una plataforma de software diseñada específicamente para revolucionar la administración de barberías y peluquerías. El propósito central de este sistema es centralizar la operatividad del negocio, conectando de forma ágil y eficiente a los profesionales del corte con su clientela a través de un entorno tecnológico moderno.

Al dejar atrás los sistemas analógicos —como las agendas de papel o la interrupción constante por llamadas telefónicas—, esta herramienta busca erradicar la ineficiencia administrativa, previniendo solapamientos en las citas y optimizando el tiempo útil de cada trabajador.

Para materializar esta visión, el proyecto persigue las siguientes metas principales:

- **Consolidación de datos:** Agrupar en un único repositorio relacional todo el volumen de información del sistema, abarcando desde el registro de locales y perfiles de empleados, hasta el histórico de reservas y el catálogo de servicios, garantizando su integridad y disponibilidad inmediata.
- **Gestión eficiente del tiempo:** Dotar a los profesionales de la barbería de un panel de control interactivo que les permita visualizar su planificación diaria, administrar su carga de trabajo y actualizar el progreso de sus citas en tiempo real.

- **Control administrativo integral:** Facilitar a los gestores la capacidad de supervisar el funcionamiento global, administrar los permisos de las cuentas de usuario, configurar nuevas sucursales y ajustar la oferta de servicios o precios con total flexibilidad.
- **Independencia del consumidor:** Brindar a los clientes la posibilidad de autogestionar sus propias reservas, ofreciéndoles la libertad de escoger su centro preferido, el servicio deseado y el profesional de su elección de manera ininterrumpida desde su dispositivo móvil.
- **Usabilidad y diseño:** Desarrollar una interfaz gráfica altamente intuitiva y atractiva que asegure una interacción fluida, minimizando la curva de aprendizaje y fomentando la fidelización del usuario final.
- **Robustez y protección:** Construir el software sobre una arquitectura distribuida que asegure la privacidad de los datos personales, implementando sistemas de autenticación fiables y un aislamiento adecuado del entorno de base de datos.

1.1. Estructura del repositorio (Backend)

El código fuente del servidor Spring Boot está organizado siguiendo los principios de separación por capas (arquitectura de N-capas). Las carpetas principales y sus responsabilidades son:

- `/src/main/java/com/darkmatter/bookcut/`
 - `config/`: Configuraciones globales de seguridad y CORS.
 - `controller/`: Controladores REST (Endpoints de la API).
 - `model/`: Entidades JPA que mapean las tablas de MySQL (`Cita`, `Usuario`, etc.).
 - `repository/`: Interfaces de Spring Data para el acceso a la base de datos.
 - `service/`: Clases con la lógica de negocio y validaciones (ej. evitar solapamiento de citas).

- [/src/main/resources/](#)
 - [db/migration/](#): Scripts SQL de Flyway para la creación de esquemas y seeders de datos iniciales
 - [application.properties](#): Credenciales de conexión a MySQL y configuración del puerto.
-

2. Arquitectura de la aplicación

2.1. Introducción

Para el desarrollo y puesta en marcha de **Book&cut** se ha prescindido de arquitecturas monolíticas tradicionales, apostando por un modelo **Cliente-Servidor** fuertemente desacoplado mediante una **API REST**. Para ello, se ha empleado el siguiente conjunto de herramientas y entornos de trabajo:

- **Entornos de desarrollo (IDE):**
 - Visual Studio Code / IntelliJ IDEA (desarrollo de lógica de servidor y aplicación móvil).
 - **Gestión de Base de Datos y Virtualización:**
 - MySQL Workbench > v8.0 (administración gráfica de datos).
 - Docker Desktop (virtualización y aislamiento mediante contenedores).
 - **Control de versiones y colaboración:**
 - Git y GitHub (control de ramas y repositorios remotos).
 - **Librerías y frameworks principales:**
 - Spring Boot > v3.x (framework principal de servidor).
 - Flutter > v3.x (SDK de desarrollo móvil).
 - Flyway (herramienta de migración y control de versiones de base de datos).
-

3. Backend

En este apartado se detalla la configuración, estructura y lógica de negocio implementada en el servidor de la aplicación Bookcut. Para garantizar un código limpio, modular y altamente mantenible, se ha optado por implementar el patrón de diseño **Controller-Service-Repository**.

3.1. Contexto Tecnológico

El núcleo del servidor se ha construido utilizando las siguientes tecnologías y herramientas de vanguardia:

- **Framework principal:** Spring Boot 4.0.3 (basado en Java 25), proporcionando un entorno robusto y optimizado para APIs REST.
- **Gestor de dependencias:** Maven.
- **Base de Datos:** MySQL 8.0. Para garantizar la consistencia entre los entornos de los desarrolladores, se ejecuta mediante contenedores **Docker**, mapeando el puerto al 3307 para evitar conflictos locales.
- **Persistencia de datos:** Spring Data JPA con Hibernate 7.2, que actúa como ORM (Object-Relational Mapping) eliminando la necesidad de escribir sentencias SQL manuales.
- **Control de versiones de BBDD:** Flyway. Esta herramienta ejecuta scripts de migración de forma automática al iniciar el servidor, asegurando que la estructura de tablas esté siempre actualizada.

3.2. Modelo (Entidades Clave)

El modelo de datos relacional se refleja en la aplicación Java a través de las siguientes entidades principales, las cuales mapean exactamente las tablas de la base de datos:

- **Usuario:** Entidad fundamental que gestiona la autenticación y la identidad en el sistema. Además de las credenciales de acceso (correo electrónico y contraseña) y los roles (CLIENTE / BARBERO), se ha extendido para incluir información personal detallada como nombre, apellidos y teléfono. Asimismo, incorpora un campo `url_foto_perfil` para la vinculación de imágenes de usuario almacenadas en el servidor.
- **Barbero:** Perfil profesional vinculado directamente a un Usuario y a una Barbería física, donde se define su especialidad de corte.

- **Servicio:** Catálogo que incluye el nombre del servicio prestado (ej. Corte, Arreglo de barba) y su tarifa asociada.
- **HorarioBarbero:** Entidad encargada de definir los rangos de disponibilidad y turnos de cada profesional.
- **Cita:** Es la entidad central que orquesta las reservas. Relaciona de forma unívoca al Cliente, el Barbero, el Servicio contratado, la fecha y hora exacta, y su estado actual (PENDIENTE, COMPLETADA o CANCELADA).

3.3. Lógica de Negocio (Servicios)

Para garantizar el correcto funcionamiento y la integridad de la información, se han implementado reglas estrictas en la capa de servicios:

- **Validación de Disponibilidad:** El servicio de gestión de reservas ([CitaService](#)) incorpora un algoritmo que impide el solapamiento de citas. Si el sistema detecta que el identificador de un barbero ya tiene un registro en la tabla de citas para una misma fecha y hora, se bloquea la operación lanzando una excepción controlada (Error 500 - *RuntimeException*).
- **Estandarización de Fechas:** Para asegurar una comunicación perfecta con la aplicación móvil (Frontend), todas las fechas y horas se procesan y serializan bajo el estándar internacional ISO 8601 ([yyyy-MM-dd'T'HH:mm:ss](#)) mediante anotaciones [@JsonFormat](#).
- **Seguridad y CORS:** Se ha configurado una política global de CORS (Cross-Origin Resource Sharing) que permite peticiones desde cualquier origen y habilita los métodos HTTP estándar (GET, POST, PUT, DELETE). Esto facilita el consumo fluido de la API desde la aplicación Flutter sin bloqueos de seguridad restrictivos por parte del cliente.

Servicio de Notificaciones (EmailService): Se ha integrado un sistema de mensajería basado en *Java Mail Sender* y el protocolo SMTP para la gestión de recuperación de cuentas. Este servicio genera y envía códigos

de verificación temporales (15 minutos de validez) para permitir el restablecimiento de contraseñas de forma segura.

- **Robustez de la Seguridad (Spring Security):** Se ha refinado la configuración de seguridad desactivando la protección CSRF para habilitar peticiones *stateless* compatibles con JWT. Además, se ha depurado el filtro de peticiones (`JwtRequestFilter`) para asegurar que el `SecurityContextHolder` se establezca correctamente incluso en peticiones complejas de tipo `multipart/form-data` (subida de imágenes), evitando errores de usuario anónimo.
-

3.4. Controlador (API Endpoints)

La interfaz de comunicación entre la aplicación móvil y la lógica del servidor se realiza a través de las siguientes rutas principales (Endpoints):

- `POST /api/citas/reservar`: Procesa la creación de nuevas citas, ejecutando previamente las validaciones de disponibilidad del barbero.
- `GET /api/citas/historial/{idUserario}`: Devuelve el listado completo y detallado de citas asociadas a un cliente específico.
- `GET /api/citas/barbero/{idBarbero}`: Recupera la agenda de trabajo diaria/semanal de un profesional concreto.
- `GET /api/barberos/todos`: Proporciona el listado completo de los barberos activos en el sistema para que el cliente pueda elegir.
- `GET /api/servicios`: Devuelve el catálogo actual de servicios y sus precios para mostrar en la interfaz de reserva.

- [GET /api/perfil](#): Recupera los datos del perfil del usuario autenticado extrayendo su identidad directamente del Token JWT.
 - [PUT /api/perfil](#): Permite la actualización de datos personales (teléfono, nombre, etc.) de forma segura.
 - [POST /api/perfil/imagen](#): Endpoint especializado en la recepción de archivos `MultipartFile` para actualizar la foto de perfil del usuario, con un límite configurado de 10MB.
 - [POST /api/auth/recuperar](#): Inicia el flujo de recuperación de contraseña enviando el código al email del usuario.
-

4. Frontend

Para el desarrollo de la capa cliente o interfaz de usuario de Book&cut, se ha apostado por un enfoque "Mobile First" orientado a ofrecer una experiencia nativa, fluida y altamente reactiva. Al contrario que las aplicaciones web tradicionales renderizadas en el servidor, este frontend actúa como una entidad independiente que se comunica de forma asíncrona con el backend.

4.1. Tecnologías y Herramientas Base

La construcción de la aplicación cliente se ha llevado a cabo utilizando las siguientes tecnologías:

- **Flutter**: Framework de código abierto desarrollado por Google. Se ha elegido por su capacidad de generar aplicaciones multiplataforma (compilando nativamente para Android e iOS) a partir de un único código base, reduciendo drásticamente los tiempos de desarrollo y mantenimiento.
- **Dart**: Lenguaje de programación tipado y optimizado para el desarrollo de interfaces de usuario (UI) utilizado por Flutter, que permite la compilación "Ahead-of-Time" (AOT) para un rendimiento nativo.

- **Librerías externas (Packages):** Se han integrado dependencias específicas para el manejo de peticiones HTTP, decodificación de archivos JSON y la gestión de la navegación interna de la aplicación.

4.2. Estructura Visual y Diseño (UI/UX)

La arquitectura de la interfaz en Flutter se basa en la composición. Todo elemento visible (e invisible) en la pantalla es un componente modular.

- **Árbol de Widgets:** La interfaz gráfica no emplea lenguajes de marcado como HTML o CSS. En su lugar, se construye mediante un árbol jerárquico de *Widgets* (componentes visuales reutilizables) que definen la estructura, el estilo y la disposición de elementos como botones, listas de barberos o formularios de reserva.
- **Material Design:** Se han implementado los patrones de diseño y las guías de estilo visual de *Material Design 3* para garantizar una experiencia de usuario estándar, intuitiva y accesible, con un sistema de colores corporativos consistente y tipografías legibles.
- **Diseño Responsivo:** Aunque la aplicación está orientada principalmente a dispositivos móviles (smartphones), la estructuración mediante *Flexbox* (columnas y filas en Flutter) permite que la interfaz se adapte correctamente a diferentes tamaños y densidades de pantalla.

4.3. Gestión del Estado y Lógica del Cliente

Una de las características clave del frontend de Book&cut es su interactividad. La aplicación distingue entre interfaces estáticas e interfaces dinámicas:

- **Widgets sin estado (Stateless):** Utilizados para elementos que no cambian una vez renderizados (por ejemplo, el logotipo de la aplicación o las etiquetas de texto de las políticas de privacidad).
- **Widgets con estado (Stateful) y Gestión de Estado:** Empleados para pantallas que deben reaccionar a la interacción del usuario. Por ejemplo, al seleccionar una fecha en el calendario de reservas, la vista se actualiza automáticamente para mostrar solo las horas disponibles del barbero, sin necesidad de recargar la pantalla completa.

4.4. Integración y Consumo de la API REST

El frontend carece de conexión directa a la base de datos MySQL por motivos de seguridad y arquitectura. Toda la información mostrada a los usuarios proviene del Backend descrito en el capítulo anterior.

- **Peticiones Asíncronas (HTTP):** La aplicación móvil utiliza métodos asíncronos para realizar peticiones (GET, POST, PUT, DELETE) a los endpoints expuestos por Spring Boot (ej. [/api/citas/reservar](#)).

Esto asegura que la interfaz gráfica nunca se quede "congelada" mientras espera la respuesta del servidor.

- **Mapeo de Datos (JSON):** Las respuestas del servidor, enviadas en formato JSON, son interceptadas y transformadas mediante modelos de datos en Dart (clases equivalentes a las entidades de Java). Esto permite que la información de Barberos, Servicios y Citas se renderice correctamente en la pantalla del dispositivo del cliente.
-

5. Documentación Técnica

5.1.1. Análisis

En esta sección se exponen de forma detallada las especificaciones y características que el sistema **Book&cut** debe cumplir para satisfacer las necesidades de las barberías modernas. Se han dividido en requisitos funcionales (las acciones que el sistema permite realizar) y requisitos no funcionales (los atributos de calidad del sistema).

5.1.1. Requisitos funcionales

Para estructurar las funcionalidades del sistema, se toman como referencia los diferentes niveles de acceso y roles de usuario definidos en la base de datos:

CLIENTE

- Iniciar sesión mediante credenciales de acceso (correo electrónico y contraseña).
- Gestión de perfil: Consultar y actualizar sus datos personales básicos.

- Gestión de citas: Solicitar una nueva reserva seleccionando al barbero de su preferencia, el servicio deseado y un tramo horario disponible.
- Visualizar el listado histórico y actual de sus citas, filtradas por su estado (Pendientes, Confirmadas, Completadas o Canceladas).
- Cancelar de forma autónoma una cita previamente solicitada.

EMPLEADO (BARBERO)

- Autenticarse en el sistema con las credenciales proporcionadas por la administración.
- Gestión de agenda: Visualizar su carga de trabajo diaria y acceder al listado de citas que los clientes han reservado específicamente con él.
- Filtrar y buscar citas en su agenda según el estado de las mismas.
- Modificar el estado de las reservas (por ejemplo, marcar un servicio como "Completado" una vez finalizado el corte).
- Visualizar las observaciones o notas que el cliente haya podido dejar al momento de la reserva.

ADMINISTRADOR

- Iniciar sesión en el panel de control del sistema.
- Gestión de usuarios: Acceder al listado completo del personal (barberos) y de la clientela registrada.
- Dar de alta nuevos perfiles profesionales de barberos y configurar sus horarios y especialidades.
- Gestión comercial: Administrar el catálogo de servicios, pudiendo crear, modificar precios o eliminar opciones de corte.
- Aplicar bajas lógicas a usuarios del sistema para inhabilitar su acceso sin destruir el histórico de datos asociado.

5.1.2. Requisitos no funcionales

Además de las acciones descritas, la plataforma ha sido diseñada para cumplir con los siguientes estándares de calidad y rendimiento:

- **Arquitectura y Rendimiento:** El sistema debe soportar múltiples peticiones concurrentes sin degradación del servicio, delegando el procesamiento intensivo al backend desarrollado en Spring Boot y minimizando el consumo de recursos en el dispositivo móvil.
- **Seguridad y Privacidad:** “*” Las contraseñas no se almacenarán en texto plano.

- Todas las comunicaciones entre la aplicación móvil (Flutter) y el servidor (Java) deben estar protegidas y validadas mediante la configuración estricta de CORS.
 - Protección activa contra vulnerabilidades de inyección SQL delegada a la capa de persistencia (Spring Data JPA).
 - **Usabilidad y Experiencia de Usuario (UX):** La interfaz móvil debe ser altamente intuitiva, aplicando los principios de *Material Design*. El usuario debe recibir retroalimentación visual inmediata (mensajes de confirmación o error) al realizar cualquier acción.
 - **Diseño Responsivo:** La aplicación en Flutter debe adaptar su renderizado correctamente a diferentes tamaños y resoluciones de pantalla de dispositivos móviles (smartphones y tablets).
 - **Mantenibilidad y Escalabilidad:** El código fuente debe respetar el patrón Controller-Service-Repository en el backend y una estructura modular de Widgets en el frontend, permitiendo la futura incorporación de nuevas funcionalidades (como pasarelas de pago) sin necesidad de reescribir el núcleo de la aplicación.
 - **Aislamiento del Entorno:** La persistencia de datos debe estar contenerizada mediante Docker, garantizando que la base de datos MySQL opere en un entorno aislado y replicable.
-

6. Desarrollo

6.1. Diseño de la BBDD - MySQL

La base de datos de **Book&cut** ha sido diseñada utilizando **MySQL 8.0** como sistema de gestión relacional, desplegado a través de contenedores Docker para garantizar la portabilidad del entorno. Todo el esquema de datos y los registros

iniciales (seeders) se gestionan mediante **Flyway**, utilizando un script de migración estructurado que automatiza la creación del esquema en el arranque del servidor.

El modelo relacional está compuesto por cinco tablas principales ([tabla_usuarios](#), [tabla_barberias](#), [tabla_barberos](#), [tabla_servicios](#) y [tabla_citas](#)), diseñadas bajo los siguientes principios fundamentales:

6.1.1. Normalización

Para evitar la redundancia de datos y las anomalías de actualización, el esquema ha sido normalizado hasta la Tercera Forma Normal (3FN). Las decisiones clave incluyen:

- **Separación de Entidades:** La información genérica de acceso (correo y contraseña) se almacena en [tabla_usuarios](#). Los detalles profesionales específicos de los barberos no ensucian esta tabla, sino que se han extraído a una tabla independiente ([tabla_barberos](#)), vinculada mediante una relación 1:1 ([identificador_usuario](#) como campo UNIQUE).
- **Catálogos independientes:** Los locales ([tabla_barberias](#)) y el catálogo de precios ([tabla_servicios](#)) son entidades fuertes. Si un servicio cambia de precio o la barbería cambia de dirección, solo se actualiza un registro, reflejándose automáticamente en todo el sistema.

6.1.2. Integridad referencial

La consistencia de la información está garantizada a nivel de motor de base de datos mediante el uso estricto de **Claves Foráneas (Foreign Keys)**.

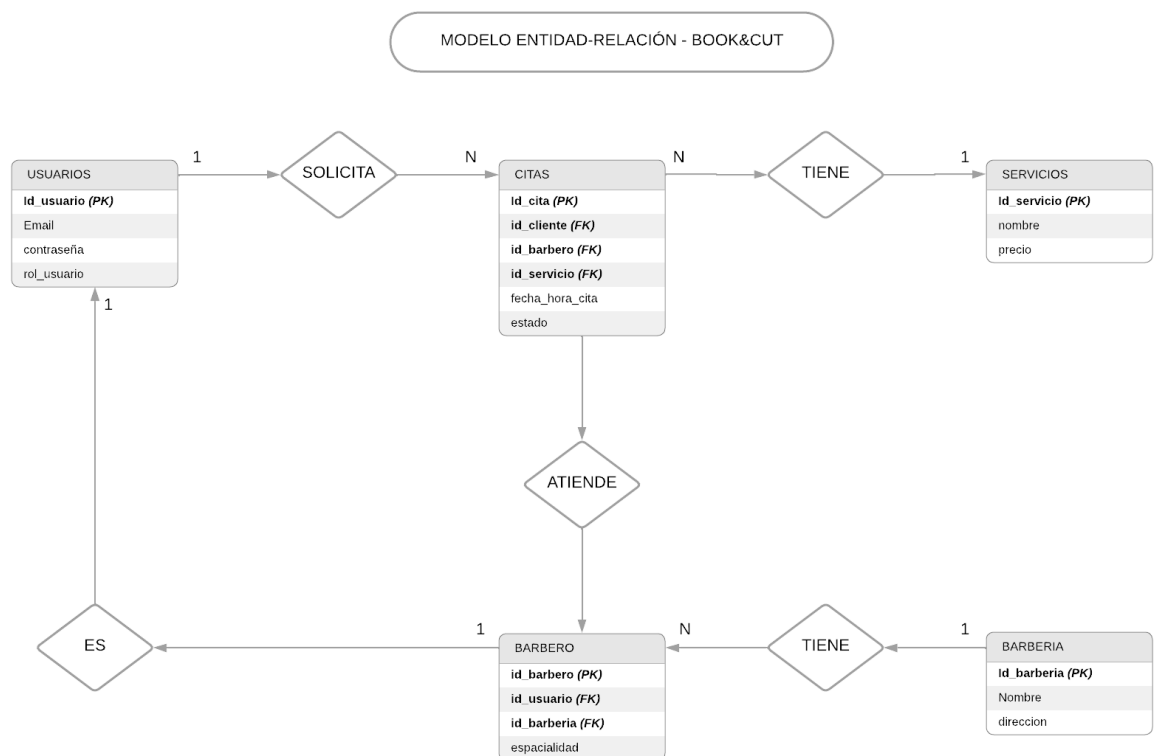
- En la [tabla_citas](#), las columnas [identificador_cliente](#), [identificador_barbero](#) y [identificador_servicio](#) son claves foráneas que referencian a sus respectivas tablas maestras.
- Esto impide, por ejemplo, que un usuario pueda reservar un servicio que no existe o que se elimine un barbero de la base de datos si este tiene citas asociadas en el histórico.
- Adicionalmente, se han utilizado restricciones de unicidad (UNIQUE) en el [correo_electronico](#) para evitar cuentas duplicadas.

6.1.3. Baja lógica y Gestión de Estados

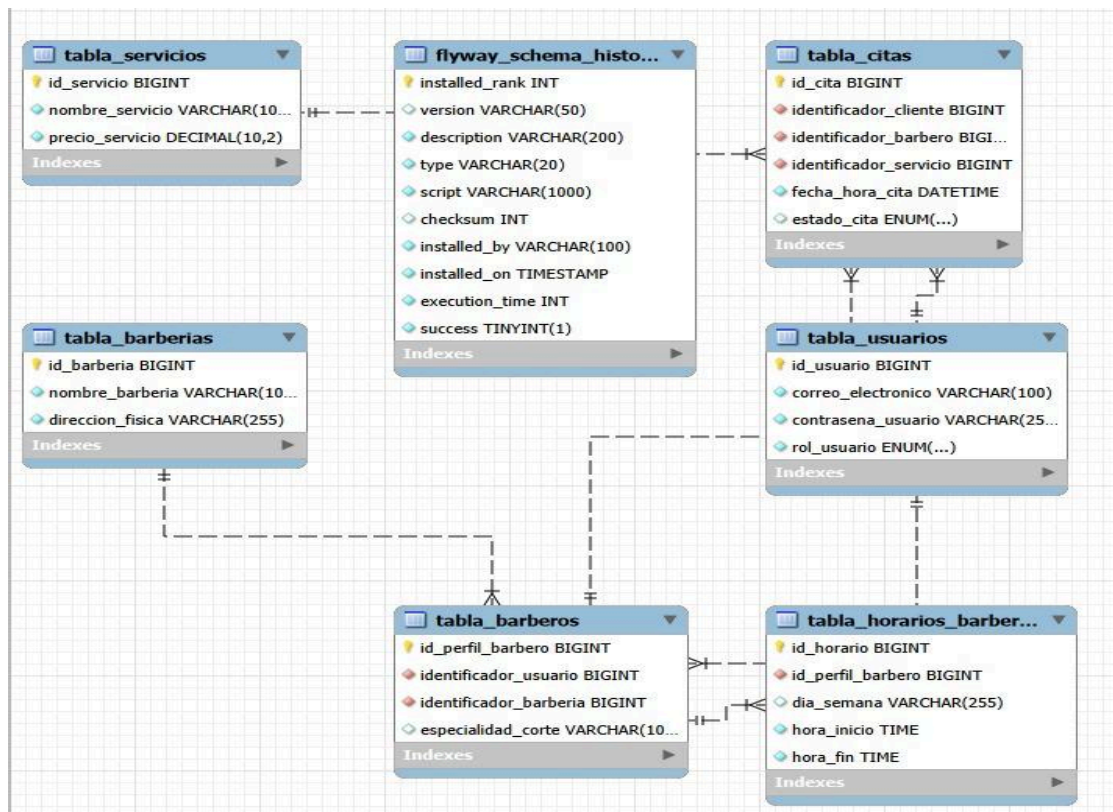
Para mantener la integridad del historial del negocio (facturación, estadísticas de citas pasadas, etc.), el sistema evita la eliminación física de registros críticos utilizando estrategias de baja lógica y estados:

- **En las citas:** No se ejecutan sentencias DELETE cuando un cliente anula una reserva. En su lugar, la tabla `tabla_citas` implementa un campo de control `estado_cita` del tipo `ENUM( ' PENDIENTE ' , ' COMPLETADA ' , ' CANCELADA ' )`. Al anular, el estado cambia a CANCELADA, liberando el tramo horario pero conservando el registro en la base de datos para futuras auditorías o estadísticas.
- **En los usuarios:** Siguiendo este mismo principio, el borrado de cuentas de clientes o barberos se gestiona desde el backend mediante suspensión temporal o inhabilitación, asegurando que las citas asociadas a ellos en el pasado sigan apuntando a un identificador válido.

7. Modelo Entidad / Relación de la BBDD



8. Modelo Relacional de la BBDD



tabla_usuarios

- id_usuario BIGINT (PK)
- correo_electronico VARCHAR(100) (UNIQUE)
- contrasena_usuario VARCHAR(255)
- rol_usuario ENUM('CLIENTE', 'BARBERO', 'ADMIN')

tabla_barberias

- id_barberia BIGINT (PK)
- nombre_barberia VARCHAR(100)
- direccion_fisica VARCHAR(255)

tabla_barberos

- id_perfil_barbero BIGINT (PK)
- identificador_usuario BIGINT (FK) -> Referencia a tabla_usuarios
- identificador_barberia BIGINT (FK) -> Referencia a tabla_barberias

- `especialidad_corte` VARCHAR(100)

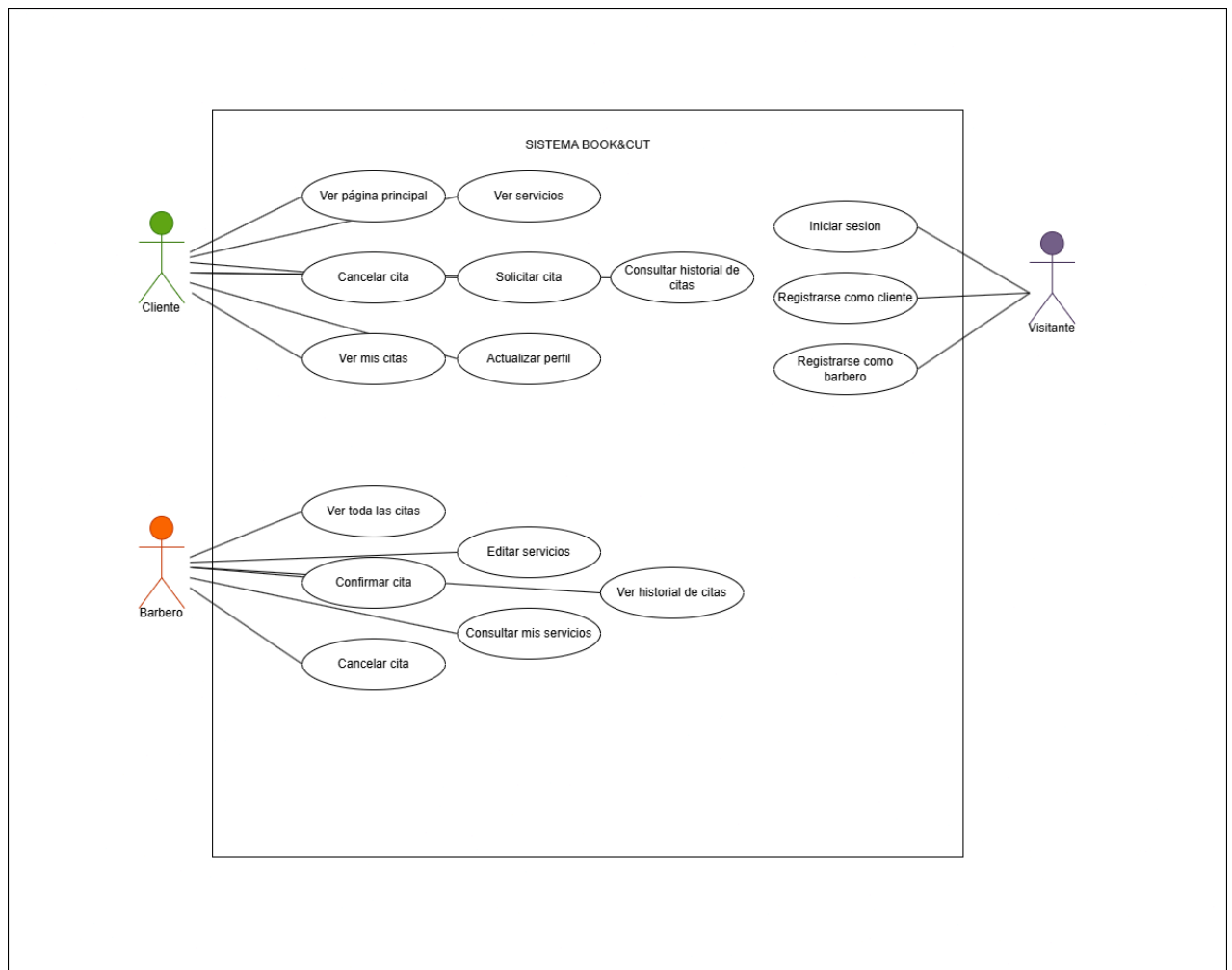
tabla_servicios

- `id_servicio` BIGINT (**PK**)
- `nombre_servicio` VARCHAR(100)
- `precio_servicio` DECIMAL(10, 2)

tabla_citas

- `id_cita` BIGINT (**PK**)
 - `identificador_cliente` BIGINT (**FK**) -> Referencia a *tabla_usuarios*
 - `identificador_barbero` BIGINT (**FK**) -> Referencia a *tabla_barberos*
 - `identificador_servicio` BIGINT (**FK**) -> Referencia a *tabla_servicios*
 - `fecha_hora_cita` DATETIME
 - `estado_cita` ENUM('PENDIENTE', 'COMPLETADA', 'CANCELADA')
-

9. Diagrama de casos de uso - Book&Cut



10. Pruebas (Validación del PMV)

Al tratarse de un Producto Mínimo Viable (PMV), la estrategia de pruebas no se ha basado en tests automatizados exhaustivos, sino en pruebas funcionales manuales orientadas a validar los flujos críticos de negocio y la correcta comunicación asíncrona entre el cliente móvil y el servidor. El objetivo es garantizar que la demostración del producto sea completamente estable y funcional.

10.1. Pruebas de Backend (API REST)

Para aislar la lógica del servidor y verificar las transacciones en la base de datos MySQL sin depender de la interfaz gráfica, se ha utilizado la herramienta **Postman**. Las pruebas ejecutadas y superadas incluyen:

- **Integridad de Endpoints:** Comprobación de que las rutas principales (como la obtención del catálogo de servicios o el registro de citas) procesan correctamente el JSON y devuelven los códigos de estado HTTP adecuados (**200 OK**, **201 Created**).
- **Validación de Reglas de Negocio:** Se han forzado escenarios de error, como intentar agendar una cita en un tramo horario donde el barbero ya figura como ocupado. Se ha comprobado que el servicio (**CitaService**) intercepta correctamente el solapamiento, aborta la transacción y devuelve un error controlado, protegiendo la integridad de la agenda.

A) Creación de una reserva exitosa: Se envía una petición **POST** al endpoint **/api/citas/reservar** con el JSON correspondiente a un cliente, un barbero y un servicio. El servidor procesa la petición correctamente, la inserta en la base de datos y devuelve un código de estado **HTTP 200 OK**, adjuntando en el cuerpo de la respuesta la cita generada con su nuevo identificador (**idCita: 4**) y el estado inicial ("PENDIENTE").

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8080/api/citas/reservar' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "clienteReserva": {
      "idUsuario": 1
    },
    "barberoAsignado": {
      "idPerfilBarbero": 1
    },
    "servicioContratado": {
      "idServicio": 1
    },
    "fechaHoraCita": "2026-06-25T11:00:00",
    "estadoCita": "PENDIENTE"
  }'
```

Request URL

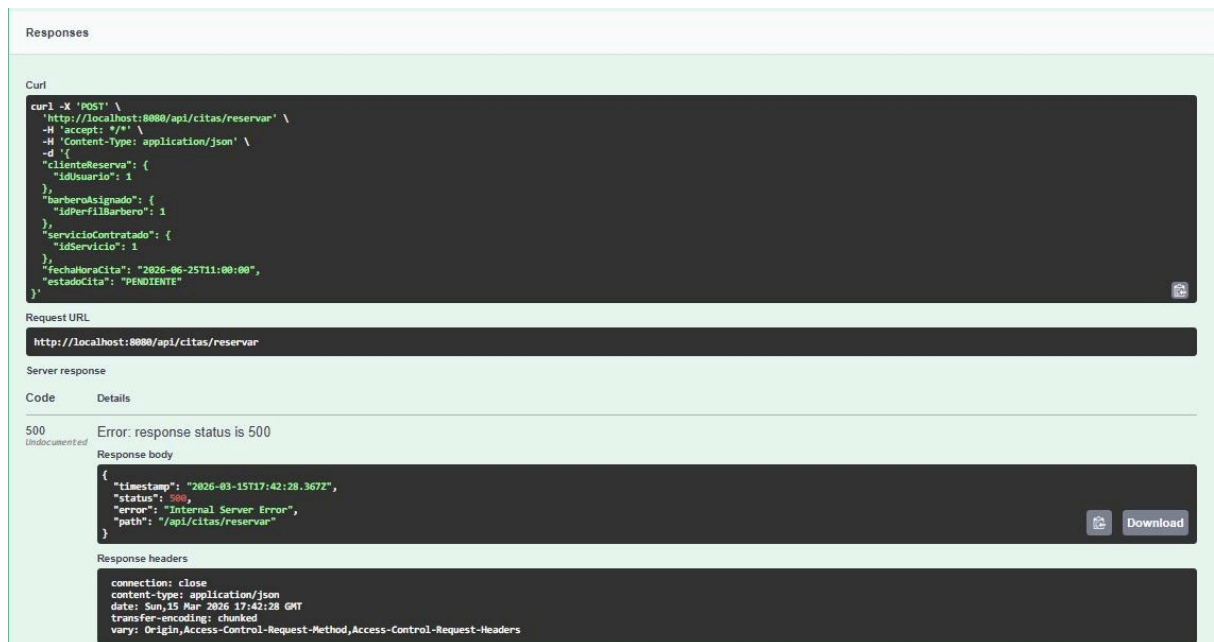
http://localhost:8080/api/citas/reservar

Server response

Code	Details
200	Response body <pre>{ "clienteReserva": { "correoElectronico": null, "contrasenaUsuario": null, "rolUsuario": null, "idUsuario": 1 }, "barberoAsignado": { "usuarioAsignado": null, "barberoAsignado": null, "especialidadCorte": null, "idPerfilBarbero": 1 }, "servicioContratado": { "nombreServicio": null, "precioServicio": null, "idServicio": 1 }, "fechaHoraCita": "2026-06-25T11:00:00", "estadoCita": "PENDIENTE", "idCita": 4 }</pre> Response headers <pre>connection: keep-alive content-type: application/json date: Sun, 15 Mar 2026 17:41:22 GMT keep-alive: timeout=60 transfer-encoding: chunked vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers</pre>

B) Validación de Reglas de Negocio (Solapamiento de citas): Para garantizar la integridad de la agenda, se fuerza un escenario de error enviando exactamente la misma petición (mismo barbero y misma fecha/hora: **2026-06-25T11:00:00**). Se comprueba que el servicio intercepta correctamente el solapamiento, aborta la transacción y devuelve un error

controlado con código **HTTP 500 (Internal Server Error)**, protegiendo así la base de datos de citas duplicadas.



10.2. Pruebas de Frontend (Interfaz en Flutter)

En la capa de presentación móvil, las pruebas se han centrado en garantizar una experiencia de usuario (UX) fluida, la prevención proactiva de errores y la correcta gestión del estado visual de la aplicación. Para ello, se han llevado a cabo las siguientes verificaciones:

- **Validación de formularios y control de errores en tiempo real:** Se ha verificado que los formularios de inicio de sesión y registro bloquean las peticiones al servidor si detectan campos vacíos, correos mal formateados o contraseñas que no cumplen los requisitos de seguridad. Al detectar una anomalía, la interfaz proporciona un *feedback* visual inmediato al usuario, resaltando los campos en rojo y desplegando notificaciones emergentes (*SnackBars*) en la parte inferior de la pantalla. Esto no solo guía al usuario, sino que protege al *backend* de recibir peticiones nulas o inválidas.



Email

cliente@prueba.com

Contraseña

Iniciar Sesión

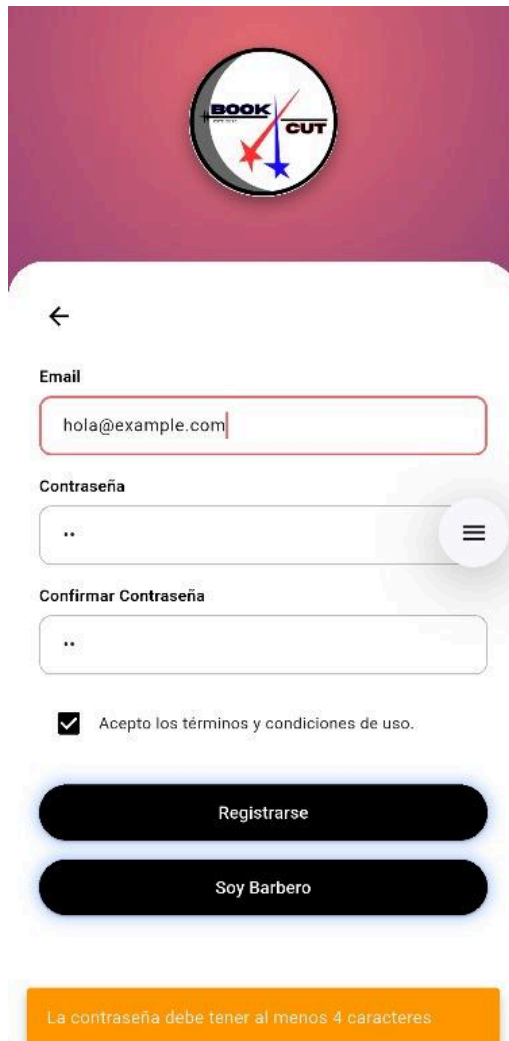
Iniciar sesion como Barbero

[¿Has olvidado la contraseña?](#)

[No estas registrado? Registrate!](#)

Por favor, rellena todos los campos

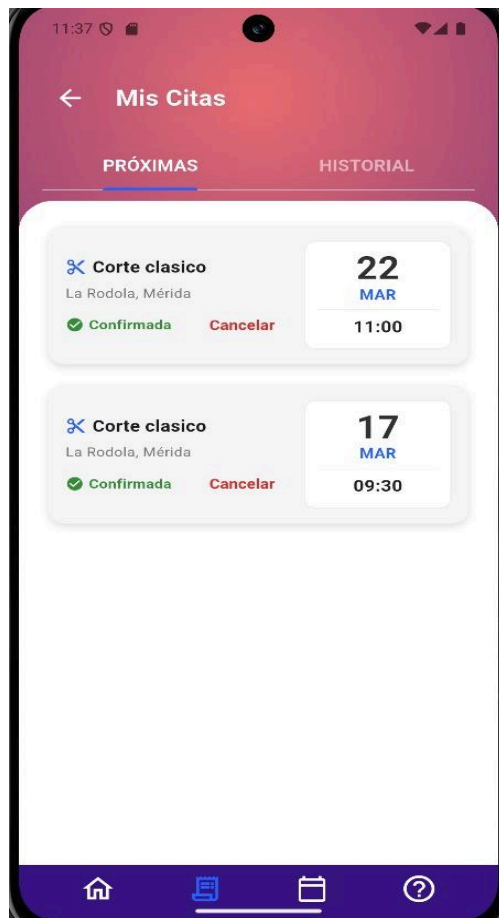
- **Extra:** Validación de contraseña



The screenshot shows a mobile application interface for a service called "BOOK CUT". At the top, there is a circular logo with the text "BOOK CUT" and a stylized red and blue star. Below the logo is a back arrow icon. The form contains the following elements:

- Email:** A text input field containing "hola@example.com".
- Contraseña:** A password input field with a toggle icon (three horizontal lines) on the right.
- Confirmar Contraseña:** A second password input field.
- Terms and Conditions:** A checkbox labeled "Acepto los términos y condiciones de uso." which is checked.
- Buttons:** Two large, rounded black buttons labeled "Registrarse" and "Soy Barbero".
- Validation Message:** An orange banner at the bottom stating "La contraseña debe tener al menos 4 caracteres".

- **Reactividad de la interfaz y sincronización de estados:** Se ha comprobado la correcta actualización de la interfaz de usuario (UI) al consumir los datos del *backend*. Por ejemplo, al cambiar el estado de una cita (cuando un Barbero la acepta, cancela o marca como "Completada"), la aplicación redibuja la pantalla del historial del cliente actualizando automáticamente las etiquetas de estado (ej. *Confirmada* en verde), sin necesidad de que el usuario reinicie la aplicación. Esto demuestra una correcta implementación de la gestión de estados en Flutter.



11. Proceso de despliegue

A diferencia de los despliegues web tradicionales que requieren la configuración manual de un servidor de aplicaciones externo (como Apache Tomcat o GlassFish), el proceso de despliegue de Bookcut se ha modernizado y simplificado mediante el uso de **contenedores** y **servidores embebidos**.

El entorno necesario para poner en marcha el Producto Mínimo Viable (PMV) se divide en tres fases: despliegue de la persistencia, ejecución del servidor y compilación del cliente.

11.1. Despliegue de la Base de Datos (Docker Desktop)

Para garantizar un entorno de ejecución idéntico en cualquier máquina y evitar conflictos de puertos con instalaciones locales previas, el motor de base de datos MySQL 8.0 se ejecuta de forma virtualizada.

1. Se requiere tener en ejecución el demonio de **Docker Desktop**.
2. Mediante un archivo de configuración ([docker-compose.yml](#)) o un comando directo de la CLI de Docker, se descarga la imagen oficial de MySQL y se levanta el contenedor.
3. **Mapeo de puertos:** El puerto interno del contenedor (3306) se expone al host local a través del puerto **3307**. Esto permite que la base de datos [base_datos_bookcut](#) esté accesible para el backend sin interferir con otros servicios locales del desarrollador.



☐	bookcut	-	-	-	0.79%	14 minutes ago			
☐	contenedor_mysql_bookcut	2bb555b2feab	mysql:8.0	3307:3306	0.79%	14 minutes ago			

11.2. Ejecución del Servidor Backend (Spring Boot)

El framework Spring Boot incorpora su propio servidor web Apache Tomcat embebido, lo que elimina la necesidad de generar archivos [.war](#) o usar plataformas externas para el PMV.

1. A través del IDE (Visual Studio Code / IntelliJ) o mediante la terminal con el gestor Maven, se ejecuta la clase principal [BookcutApplication.java](#).
2. Durante la secuencia de arranque, Spring Boot detecta la dependencia de **Flyway** e inyecta automáticamente los scripts SQL ([V1__crear_tablas_iniciales.sql](#) y [V2__crear_tabla_horarios.sql](#)) en el contenedor de Docker, creando la estructura relacional y los *seeders* sin intervención manual.
3. El servidor queda a la escucha (Listening) procesando peticiones a través del puerto **8080** en [localhost](#).

- **Pasarela de pago:** La reserva se realiza de forma "simulada" sin exigir un pago por adelantado, ya que la integración con plataformas como Stripe o PayPal queda fuera del alcance de este PMV, aunque el sistema está preparado para su futura implementación en la capa de servicios.

12.2. Próximos pasos

Para completar la aplicación de cara a la defensa final del proyecto, el equipo tiene planificado:

- Implementar el panel de control completo para el rol **ADMIN**, permitiendo la gestión gráfica del catálogo de servicios y la facturación.
- Activar Spring Security con **Tokens JWT** (JSON Web Tokens) para una autenticación mucho más robusta y segura entre Flutter y el Backend.
- Desarrollar un sistema de notificaciones por email al registrar una cita.

13. BIBLIOGRAFÍA

13.1. Documentación Oficial

- **Spring Framework & Spring Boot:**
 - Spring Boot Reference Guide. Recuperado de: <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/>
 - Spring Data JPA Reference Documentation. Recuperado de: <https://docs.spring.io/spring-data/jpa/docs/current/reference/html/>
- **Flutter & Dart:**
 - Flutter Documentation: The complete reference. Recuperado de: <https://flutter.dev/docs>
 - Dart Language Tour: Introducción al lenguaje Dart. Recuperado de: <https://dart.dev/language>
 - Material Design 3 Guidelines: Estándares de diseño de interfaces. Recuperado de: <https://m3.material.io/>
- **Base de Datos y Herramientas:**
 - MySQL 8.0 Reference Manual. Recuperado de: <https://dev.mysql.com/doc/refman/8.0/en/>
 - Docker Documentation: Referencia de contenedores y Docker Compose. Recuperado de: <https://docs.docker.com/>
 - Flyway Documentation: Migraciones de bases de datos. Recuperado de: <https://flywaydb.org/documentation/>

13.2. Webs de referencia, Tutoriales y Comunidades

- **StackOverflow:** Consultas recurrentes sobre resolución de errores en Spring Boot (CORS, Hibernate) y gestión de estado en Flutter (StatefulWidgets). Recuperado de: <https://stackoverflow.com/>

- **Baeldung:** Artículos y tutoriales avanzados sobre la implementación de arquitectura Controller-Service-Repository y configuración de Spring Data JPA. Recuperado de: <https://www.baeldung.com/>
- **Postman Learning Center:** Guías para la creación y testeo de colecciones API REST. Recuperado de: <https://learning.postman.com/>